Build Your Own Language Model (LLM) Project

Introduction

A language model is a model that assigns probabilities to sequences of words meaning the model predicts the next word given the word that came before. Repeating this prediction lets the model generate new text. Large models such as GPT-2 which we use in the project are pre trained on large amount of text and can be fine tuned to a specific domain or task with much less data. This project shows fine tuning the pre trained GPT-2 model on hotel reviews so that it learns to write in the style of a real hotel review. The model was evaluated based off perplexity and next word accuracy and used to generate reviews from prompts. I also fine tuned a second copy of GPT-2 to classify reviews as positive or negative.

Data Description

The dataset is found on Kaggle: 515K Hotel Reviews Data in Europe (https://www.kaggle.com/datasets/jiashenliu/515k-hotel-reviews-data-in-europe). The dataset is a 238 MB CSV file with 515k rows and 17 columns covering details such as hotel name, address, nationality, review date, score, etc. From the available columns in the dataset only Positive_Review and Negative_Review were used to train the model.

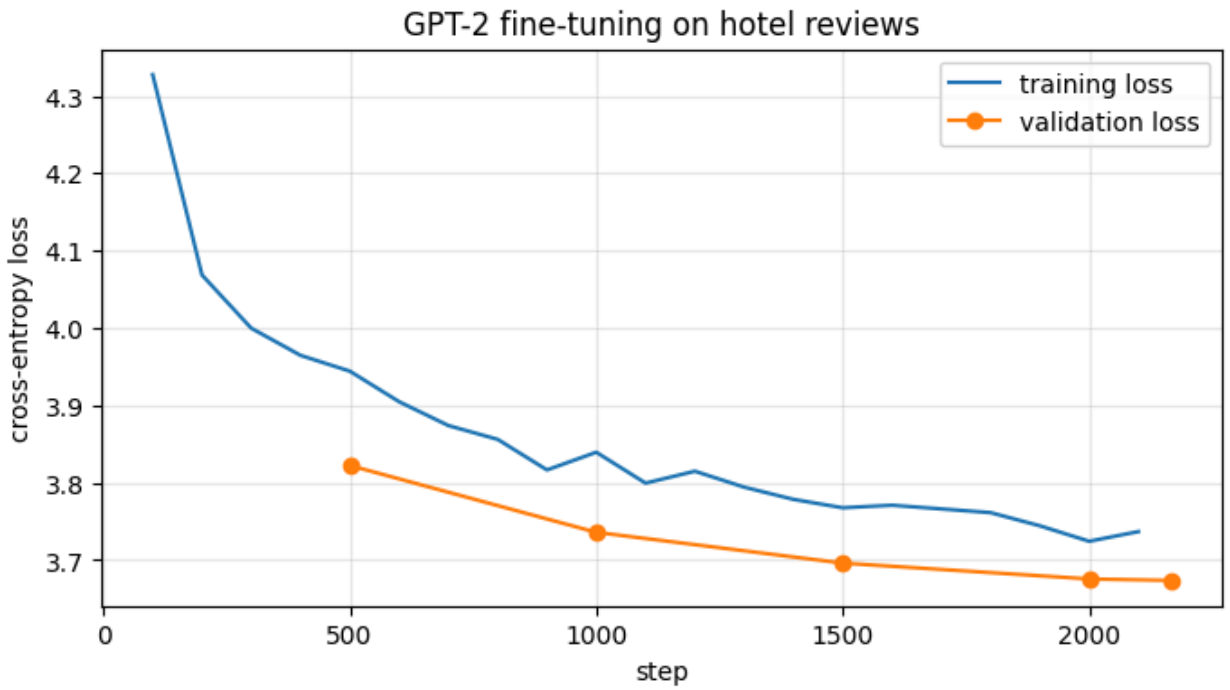
Methodology

For preprocessing we lowercase, remove punctation, tokenize, and have stop word removal implemented but chose not to apply it to the training text since the model needs words removed by the stop word removal such as “the”, “and”, etc. to produce grammatically correct sentences. Cleaned reviews were split 90/10 into training and validation set with train_test_split and saved as text files.

GPT-2 is chosen because GPT-2 is trained to predict the next word and can be measured with perplexity and used to generate text. Fine tuning a pretrained GPT-2 is more effective than training from scratch since 94 mb of text is small compared to the amount the GPT-2 was originally trained on.

The training process used Hugging Face Trainer on Google Colab’s T4 GPU. Training loss was logged every 100 steps and validation loss was logged every 500 steps. One epoch took 2167 steps and a final model was saved for later use. For testing, 4 experiments were ran on a 50k review subset, changing learning rate, block size, and model size.

Results



Both training and validation loss decreased throughout training. Biggest improvement happened in the first ~500 steps but validation loss stayed falling till the end. Final training loss was 3.74 and validation loss was 3.67.

Metric	Original GPT-2	Fine tuned GPT-2
Validation loss	5.21	3.67
Perplexity	183.68	39.39
Next word accuracy	18%	29.2%

The fine tuned model out performed the original model in every metric: validation loss dropped from 5.21 to 3.67, perplexity fell by 79% (183.68 to 39.39), and next-word accuracy rose from 18% to 29.2%. This shows that one epoch of training on hotel reviews taught the model the vocabulary and style of the dataset, which the original GPT-2 had never seen.

Experiment	Perplexity
Baseline	43.60
Higher learning rate	40.34
Shorter Context	42.18
Smaller model	53.13
More reviews	39.39

From the experiments, training on more reviews gave the best results, followed by doubling the learning rate. Smaller model was about 22% worse.

The fine tuned model was given 5 prompts and since each output are long, shorter excerpts of each output are shown below instead. For full length refer to the notebook.

```
PROMPT: the room was
  Generated Text 1: the room was too small no space to put the blankets
and had to close the door to reach the window at night although the bed
was soft there was a gap in the shower which made it difficult to shower
again it was also very uncomfortable for us to have a bath on and we had
to sleep on the floor opposite of...

PROMPT: the staff were
  Generated Text 1: the staff were superb the location was excellent and
the rooms were very quiet the staff were very helpful and friendly the
room was very clean and the pool is very good the hotel was clean and well
presented with the best views of the city we…

PROMPT: breakfast was
  Generated Text 1: breakfast was fantastic and the staff were very
helpful and friendly the location is perfect for the airport and the tram
stop is a short walk away the pool is really great and the staff were
super helpful and friendly the staff were really helpful…

PROMPT: the location of the hotel
  Generated Text 1: the location of the hotel in particular was amazing as
was the breakfast and the service which was great the hotel was a 2 minute
walk from oxford street and a good 10 minute walk to the oxford road
station which was a lovely change of pace from…

PROMPT: would not recommend because
  Generated Text 1: would not recommend because of the price quality of
food and service but the service was excellent the room was clean with
```

```
good facilities and a view of the canal and the underground tube was
lovely the staff were friendly and the hotel was very…
```

Outputs were pretty coherent and read like real hotel reviews. Sentiments usually match the prompt for example "breakfast was" and "the staff were" produced positive reviews why "the room was" produced a negative review. Some problems were repetition where "the staff were very helpful and friendly" show up in multiple prompts and contradiction happened such as "the bathroom was also very small and it was very big". The prompts also don't have a clear ending since the training text has no punctuation and the generated text were long, leading to outputs running several reviews together instead of stopping when a review is finished.

For extra credit, a second copy of GPT-2 was fine tuned with GPT2ForSequenceClassification to label reviews as positive or negative. A balanced sample of 40k reviews was split into 36k for training and 4k for validation. The model was trained for one epoch and below are the results:

Metric	**Score**
Accuracy	0.940
Precision	0.940
Recall	0.941
F1	0.940
Validation loss	0.236

Evaluation

The model's strengths include:
Measurable improvement: Perplexity fell by 79% and next word accuracy increased by 11%. One epoch allowed the model to learn format and vocab.
No overfitting: validation loss decreased through training and never increased, meaning the model learned general patterns instead of memorizing training data.
Good generated text: Generated text generally read close to real reviews and most used typical phrases seen in real reviews. Sentiment initially generally matched prompt would be stray sometimes after the beginning.

Some weaknesses include:
Repetition: long outputs led to repeated phrases since generation ended up picking the same high probability phrases.
Sentiment drift: as mentioned above, sentiment drift would be an issue as seen in one output where the prompt "would not recommend because" starts negatively but turns into a review "i

would definitely recommend this hotel to anyone”. This could be due to positive phrasing being common in the data, leading the model to draft back towards it.
Contradiction: contradiction can be seen at the end of one of the outputs: "the bathroom was also very small and it was very big", showing that the model learned how reviews sound but not the facts about these hotels.
No clear ending: since preprocessing removed punctuation, long outputs led to running several reviews together instead of stopping when a review is finished.

Conclusion

Finetuning GPT-2 on 100k hotel reviews led to a 79% reduction in perplexity and an increase from 18% to 29.2% in next word accuracy. The model generated semi realistic on topic reviews but had issues such as repetition due to the long outputs and drifts in sentiment. Experiments show that increasing training data helps most, followed with higher learning rate. A smaller model performed worse.

Possible improvement is most obviously training on more available reviews and training more than one epoch. Keeping capitalization and punctuation in the training text helps the model learn sentence structure and helps the model learn when a review is finished. Trying a larger model should improve as well since the smaller model performed worse in the experiment. Lastly using shorter outputs during generation should help reduce repetition and sentiment drift.